Week3_Day2_Hands_On_Koduru_Ushasree

Level-1 and 2 Problem: Create Database **EventDb** and create tables as per schema given below.

Requirements

1.Add entity class **UserInfo** and add public properties:

Property Name	Type	Constraint
EmailId	varchar	**PK**
UserName	varchar	Not Null, Length: 1 to 50 char.
Role	varchar	Not Null, (Admin,Participant)-CHECK
Password	varchar	Not Null Length: 6 to 20 char

2. Add entity class **EventDetails** and add below public properties.

Property Name	Type	Constraint
EventId	int	**PK**
EventName	varchar	Not Null, Length: 1 to 50 char.
EventCategory	varchar	Not Null, Length: 1 to 50 char.
EventDate	datetime	Not Null
Description	varchar	Nullable
Status	varchar	Active or In-Active, CHECK

4. Add entity class **SpeakersDetails** and add below public properties.

Property Name	Type	Constraints
SpeakerId	int	**PK**
SpeakerName	varchar	Not Null, Length: 1 to 50 char.

5. Add entity class **SessionInfo** and add below public properties.

Property Name	Type	Constraints
SessionId	int	**PK**
EventId	int	Not Null **(FK)**
SessionTitle	varchar	Not Null, Length: 1 to 50 char.
SpeakerId	int	Not Null **(FK)**
Description	varchar	Nullable
SessionStart	datetime	Not Null (Time Slot)
SessionEnd	datetime	Not Null (Time Slot)
SessionUrl	varchar	

6. Add entity class **ParticipantEventDetails** and add below public properties.

Property Name	Type	Constraints
Id	int	**(PK)**
ParticipantEmailId	varchar	Not Null **(FK)**
EventId	int	Not Null **(FK)**
SessionId	Int	Not Null **(FK)**
IsAttended	bit	Yes or No, CHECK

Technical Constraints

- Use SQL Server.
- Use appropriate data types (INT, VARCHAR, DATE, DECIMAL).
- Follow normalization up to 2NF.
- Use ANSI SQL syntax only.

Expectations:

- Proper table structure with relationships.
- Valid constraints implementation.
- Correct data insertion and retrieval queries.

Code :-

```sql
--creating EventDb Databse
CREATE DATABASE EventDb;

-- using the database
USE EventDb;
GO

-- creating UserInfo Table
CREATE TABLE UserInfo (
  EmailId varchar(50) PRIMARY KEY, -- UNIQUE and NOT NULL

  UserName varchar(50) NOT NULL ,

  Role varchar(20) NOT NULL CHECK(Role IN ('Admin','Participant')), -- checking whether Admin or Participant

  Password varchar(20) NOT NULL CHECK(LEN(Password) BETWEEN 6 AND 20) -- checking length of password must be b/w 6 and 20
);

-- creating EventDetails Table
CREATE TABLE EventDetails (
    EventId INT PRIMARY KEY,

    EventName VARCHAR(50) NOT NULL CHECK (LEN(EventName) BETWEEN 1 AND 50),-- checking length of event name

    EventCategory VARCHAR(50) NOT NULL CHECK (LEN(EventCategory) BETWEEN 1 AND 50), -- checking length of event category

    EventDate DATETIME NOT NULL,

    Description VARCHAR(255) NULL,

    Status VARCHAR(20) NOT NULL CHECK (Status IN ('Active','In-Active')) -- checking status whether it's Active or In-Active
);
```

```sql
-- creating SpeakersDetails Table
CREATE TABLE SpeakersDetails (
    SpeakerId INT PRIMARY KEY,
    SpeakerName VARCHAR(50) NOT NULL);

-- creating SessionInfo Table
CREATE TABLE SessionInfo (
    SessionId INT PRIMARY KEY,
    EventId INT NOT NULL,
    SessionTitle varchar(50) NOT NULL,
    SpeakerId INT NOT NULL,
    Description varchar(100),
    SessionStart DATETIME NOT NULL,
    SessionEnd DATETIME NOT NULL,
    SessionUrl varchar(100)

    CONSTRAINT FK_Session_Event
        FOREIGN KEY(EventId) REFERENCES EventDetails(EventId), -- Foreign Key to ensure each session belongs to an existing event

    CONSTRAINT FK_Session_Speaker
        FOREIGN KEY(SpeakerId) REFERENCES SpeakersDetails(SpeakerId) -- Foreign Key to ensure each session is assigned to a valid speaker
);

-- creating ParticipantEventDetails Table
CREATE TABLE ParticipantEventDetails (
    Id INT PRIMARY KEY,
    ParticipantEmailId varchar(50) NOT NULL,
    EventId INT NOT NULL,
    SessionId INT NOT NULL,
    IsAttended BIT NOT NULL CHECK(IsAttended IN(0,1)) -- Indicates whether the participant attended the session (1 = Yes, 0 = No)
);
```

Output :-

100 % No issues found

Messages

```
Commands completed successfully.

Completion time: 2026-03-03T21:52:11.9254676+05:30
```

Code :-

```sql
-- inserting values into UserInfo Table
INSERT INTO UserInfo VALUES
('admin@gmail.com', 'AdminUser', 'Admin', 'adminuser'),
('ravi@gmail.com', 'Ravi', 'Participant', 'ravixyz');

-- inserting values into EventDetails Table
INSERT INTO EventDetails VALUES
(1,'Tech Talks', 'Technology', '2026-03-16', 'Talk about Trending Tech Stacks', 'Active' ),
(2, 'Workshop', 'IT', '2026-02-22', 'Training regarding some useful techniques', 'In-Active');

-- inserting values into SpeakersDetails Table
INSERT INTO SpeakersDetails VALUES
(101, 'Dr.Sharma'),
(102, 'Rama Dutt');

-- inserting values into SessionInfo Table
INSERT INTO SessionInfo VALUES
(1001,1, 'AI Trends', 101, 'Discussion on AI', '2026-03-16 10:00:00', '2026-03-16 12:00:00', 'http://sessionlink.com');

-- inserting values into ParticipantEventDetails Table
INSERT INTO ParticipantEventDetails VALUES
(1,'ravi@gmail.com', 1, 1001, 1);
```

Output :-

100 % No issues found

Messages

```
Commands completed successfully.

Completion time: 2026-03-03T21:52:11.9254676+05:30
```

Code :-

```sql
-- retrieving the UserInfo detailss
select * from UserInfo;
```

Output :-

100 % No issues found

Results Messages

	EmailId	UserName	Role	Password
1	admin@gmail.com	AdminUser	Admin	adminuser
2	ravi@gmail.com	Ravi	Participant	ravixyz

Code :-

```
-- retrieving the EventDetails
select * from EventDetails;
```

Output :-

100 % No issues found

Results Messages

	EventId	EventName	EventCategory	EventDate	Description	Status
1	1	Tech Talks	Technology	2026-03-16 00:00:00.000	Talk about Trending Tech Stacks	Active
2	2	Workshop	IT	2026-02-22 00:00:00.000	Training regarding some useful techniques	In-Active

Code :-

```
-- retieving the SpeakersDetails
select * from SpeakersDetails;
```

Output :-

100 % No issues found

Results Messages

	SpeakerId	SpeakerName
1	101	Dr.Sharma
2	102	Rama Dutt

Code :-

```
-- retreiving the SessionInfo Details
select * from SessionInfo;
```

Output :-

100 % No issues found

Results Messages

	SessionId	EventId	SessionTitle	SpeakerId	Description	SessionStart	SessionEnd	SessionUrl
1	1001	1	AI Trends	101	Discussion on AI	2026-03-16 10:00:00.000	2026-03-16 12:00:00.000	http://sessionlink.com

Code :-

```
100
101    -- retreiving the ParticipantEventDetails
102    select * from ParticipantEventDetails;
```

Output :-

100 % No issues found

Results Messages

	Id	ParticipantEmailId	EventId	SessionId	IsAttended
1	1	ravi@gmail.com	1	1001	1

Technical Requirements :-

The EventDb database must be created using ANSI SQL syntax in Microsoft SQL Server, implementing a normalized relational schema up to Second Normal Form (2NF). The system should consist of five entity tables: UserInfo, EventDetails, SpeakersDetails, SessionInfo, and ParticipantEventDetails. Each table must define appropriate primary keys to enforce entity integrity and foreign key constraints to maintain referential integrity between related tables. Domain integrity must be ensured using proper data types (INT, VARCHAR, DATETIME, BIT) along with NOT NULL and CHECK constraints for validation of length, allowed values, and logical conditions (e.g., Role restricted to Admin or Participant, Status restricted to Active or In-Active, and IsAttended restricted to 0 or 1). The SessionInfo table must reference EventDetails and SpeakersDetails via foreign keys, while ParticipantEventDetails must reference UserInfo, EventDetails, and SessionInfo to model participation relationships. All relationships should follow correct cardinality rules and avoid partial dependencies to satisfy 2NF. The implementation must support valid data insertion and retrieval using SELECT and JOIN queries while preventing invalid or inconsistent data through enforced constraints.